

# RagAgent — Interview Prep Guide

Sep 27, 2026 · @krishna

## The 30-Second Pitch

### The 30-Second Pitch

Say this first, verbatim, then let follow-up questions pull you into whichever layer the interviewer cares about:

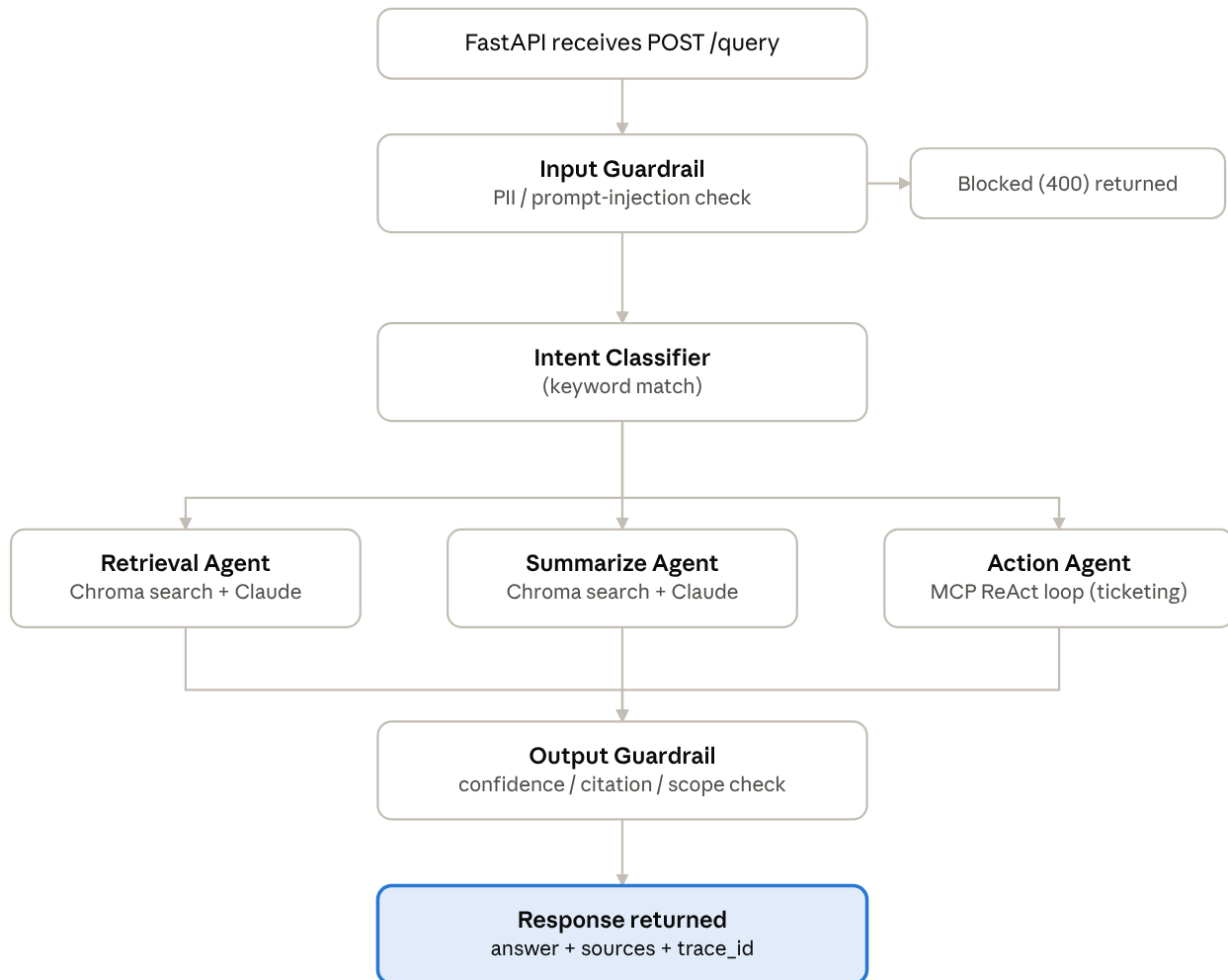
"It's a document Q&A agent for an enterprise: it retrieves relevant text from a vector database, answers using an LLM, and can also take real actions like creating support tickets — all orchestrated as a graph with safety checks on input and output, and full tracing so you can debug exactly what happened on any request."

Why this ordering works: it names the *what* (document Q&A + actions), the *how* (retrieval, orchestration graph), and the *why it's enterprise-grade* (safety checks, tracing) in one breath. An interviewer will almost always latch onto one of those three clauses and ask you to go deeper — that's your cue for which section below to draw from.

# Architecture: The Request Lifecycle

## Architecture: The Request Lifecycle

### Request lifecycle: FastAPI to response



Request lifecycle — 7 steps, one branch

Every arrow in that picture is also a recorded span (local tracer always; LangSmith too once tracing is enabled), so any request can be replayed step-by-step through `GET /traces/{trace_id}` or in the LangSmith UI. The graph is a `langgraph.StateGraph` over a single `RouterState TypedDict` — each box above is a node, each arrow an edge (some conditional, based on `_classify_intent()`).

## Design Decisions: Why Each Tool

### Design Decisions: Why Each Tool

For every tool, be ready to say what it replaced and what you gave up by choosing it — that's the question behind "why did you use X."

**LangGraph, not a hand-rolled while-loop.** A single agent that decides retrieval vs. summarize vs. action by re-prompting itself is slower, harder to trace, and non-deterministic in its routing. LangGraph makes the control flow an explicit `StateGraph`: nodes are plain functions, edges (including conditional ones) are declared up front, and the whole thing compiles into something you can visualize, unit-test node-by-node, and resume mid-graph. Trade-off: more upfront structure than "just call the API in a loop" — worth it once you have more than one path through the system.

**ChromaDB, not FAISS or Pinecone.** FAISS is a library, not a database — no metadata filtering, no persistence story of its own, you'd build that layer yourself. Pinecone is a managed service — great at scale, but it's a network call and a monthly bill for a project that needs to run entirely on a laptop with no cloud account. Chroma gives you a real client/server API and metadata filtering, but can also run embedded (in-process, writing to a local folder) with zero infrastructure — exactly the two modes this project demonstrates ( `chroma_host` set vs. unset in `config.py` ). Trade-off: Chroma is less battle-tested at very large scale than Pinecone or a hosted pgvector.

**MCP (Model Context Protocol), not a hardcoded function-calling schema.** Anthropic's function-calling API works fine when your tools live in the same codebase as your agent. MCP matters the moment tools live somewhere else — a ticketing system, a CRM, a separate team's service — because it standardizes tool discovery and invocation as a protocol, not a per-agent integration. This project's `ticketing_server.py` is a real, separately-runnable MCP server (stdio transport); the action agent discovers its tools ( `create_ticket`, `get_ticket`, `list_tickets`, `update_ticket_status` ) at runtime via `MultiServerMCPClient` rather than having them wired in by hand. That's the FDE-relevant point: in a client engagement you rarely own the systems you need to act on, and MCP is the interface layer for that.

**Guardrails as a separate layer, not prompt engineering.** "Just tell the model to refuse PII" is not auditable, not testable, and silently degrades as prompts evolve. This project's guardrails are plain Python — a `GuardrailPolicy` dataclass plus regex/keyword checks that run *before* the LLM sees the input and *after* it produces an output (confidence

thresholding, citation requirement, blocked-topic scoping). They're deterministic, unit-testable in isolation, and they produce a structured `allowed: true/false` verdict the API can act on — not a vibe.

**Two observability layers, not one.** LangSmith is excellent but it's a third-party SaaS dependency — if a client's security team won't allow it, or the account isn't configured, you still need to explain what happened on a given request. So there's a lightweight, dependency-free tracer ( `Tracer / Span / Trace` dataclasses, context-manager based) that writes structured JSON and backs the `/traces/{trace_id}` endpoint regardless of LangSmith. LangSmith is layered on top, toggled purely by environment variables ( `LANGSMITH_TRACING` , `LANGSMITH_API_KEY` , `LANGSMITH_PROJECT` ) with no code-level instrumentation required — it traces automatically once those envs are set. Trade-off: maintaining two tracing paths is more code than picking one, but it's the difference between a demo and something that survives a client's compliance review.

**FastAPI, not Flask.** Async-native (matters once you're awaiting LLM calls and MCP tool calls), Pydantic schemas give you request/response validation for free, and the auto-generated Swagger UI at `/docs` doubles as a demo interface with zero extra work.

## Debugging War Stories (Behavioral Interview Material)

## Debugging War Stories (Behavioral Interview Material)

These are real bugs hit while bringing this project up end-to-end — use them for "tell me about a time you debugged something tricky." Each is framed STAR-style.

### Story 1 — the relative-path bug that made retrieval silently return nothing

- *Situation:* `scripts/ingest.py` , run from the repo root, populated the Chroma collection successfully. But once the API server was started with `cd src && python -m uvicorn ...` , every query answered "I don't have any ingested documents to answer from yet" — no error, just an empty-feeling answer.
- *Task:* Figure out why data that was clearly ingested wasn't visible to the running server.
- *Action:* Rather than guessing, printed `settings.chroma_persist_directory` from both entry points. Ingest resolved the relative `data/vectorstore` against the repo root ( `RagAgent/data/vectorstore` ); uvicorn, launched from `src/` , resolved the *same relative string* against its own CWD ( `RagAgent/src/data/vectorstore` — empty). Two processes, two different working directories, one relative path.

- *Result:* Rewrote the settings loader's path resolution to always anchor relative paths to the project root (computed once from `__file__`, not CWD) rather than trusting `os.getenv()` values as-is. Lesson for the interview: config that depends on "the directory you happened to launch from" is a production bug waiting to happen — anchor to something invariant.

## Story 2 — the dependency that was installed but never actually used

- *Situation:* After fixing the path bug, the very next run failed with `TypeError: Anthropic authentication failed: no API key or authorization credentials were provided`, even though the key was correctly set in `.env`.
- *Task:* Explain why an env var that was definitely in the file wasn't reaching the process.
- *Action:* Checked `requirements.txt` — `python-dotenv` was listed and installed. Grepped the codebase for `load_dotenv` and found zero matches anywhere. The dependency existed on disk but no line of code ever called it, so `.env` was never actually read into the process environment — only variables already in the shell's environment were visible.
- *Result:* Added `load_dotenv(_RAGAGENT_ROOT / ".env")` at the top of `config.py`, before `Settings` reads any `os.getenv()` calls. Lesson: "it's in requirements.txt" and "it's actually wired up" are different claims — verify the second one, especially for anything that silently no-ops instead of raising.

## Story 3 — the PATH bug that pointed at the wrong interpreter despite an active venv

- *Situation:* With `(.venv)` visibly active in the prompt, `uvicorn ragagent.api.app:app --reload` still failed with `ModuleNotFoundError: No module named 'langchain_anthropic'` — a package that was definitely installed in that same venv.
- *Task:* Determine why an active virtual environment wasn't being honored.
- *Action:* Ran `which uvicorn` and `which python` side by side. `python` correctly resolved inside `.venv/bin/`; `uvicorn` resolved to a stray global install from `/Library/Frameworks/Python.framework/...` that had been installed before the venv existed and was still earlier on PATH for that specific executable name.
- *Result:* Switched the run command to `python -m uvicorn ragagent.api.app:app --reload`, which forces Python to resolve `uvicorn` as a module through the currently-active interpreter rather than trusting whichever `uvicorn` binary PATH finds first. Lesson: "the venv is active" doesn't guarantee every command on your PATH is the venv's version — `python -m <tool>` is the safer default for exactly this reason.

## Anticipated Interview Q&A

### Anticipated Interview Q&A

**Q: Why LangGraph instead of just calling the Anthropic API in a loop?** A: Because there's real branching — retrieval vs. summarize vs. action — and a loop hides that structure inside prompt logic. A graph makes the routing explicit, testable per-node, and visualizable, which matters once more than one engineer has to reason about it.

**Q: How do you handle a wrong or hallucinated answer?** A: Two lines of defense. First, the retrieval agent's confidence score (derived from vector similarity distance) feeds the output guardrail's `min_confidence_to_auto_answer` threshold — below it, the answer isn't auto-served. Second, `require_citation` in the guardrail policy means an answer without a source chunk backing it can be rejected outright. Neither is perfect — confidence-from-distance is a proxy, not a correctness guarantee — but it's a deterministic, auditable check rather than hoping the model self-corrects.

**Q: How would you take this to production / what's missing?** A: Be honest here rather than oversell it. Missing: authentication/authorization on the API, per-tenant data isolation in Chroma (right now it's one shared collection), retry/backoff around the LLM and MCP calls, a real eval harness with labeled queries (there's a `scripts/evaluate.py` stub, not a full suite), and horizontal scaling of the vector store (embedded Chroma is single-process; the remote/server mode is the path to a shared, scalable store). What's already production-shaped: the guardrail layer, the dual observability story, and treating tools as MCP servers rather than hardcoded functions.

**Q: Why ChromaDB instead of Pinecone or a hosted vector DB?** A: Zero-infrastructure development — embedded mode needs no account, no network call, no bill — while the exact same code path supports a real client/server deployment by setting `CHROMA_HOST / CHROMA_PORT`. Pinecone is a fine choice at real scale; for a project that has to run on a laptop and demonstrate both modes, Chroma was the better fit.

**Q: What happens if the MCP (ticketing) server crashes or isn't running?** A: The action agent's tool discovery ( `MultiServerMCPClient` ) fails at the point it tries to load tools, which surfaces as an error the API can return rather than a silent no-op — because the ticketing server is a separate process communicating over stdio, its failure is a distinct, observable event, not something masked inside the agent's own process.

**Q: How does tracing actually work — walk me through it.** A: Two independent layers. A custom `Tracer / Span / Trace` set of dataclasses wraps each node in the graph as a context manager, recording start/end/metadata as structured JSON, independent of any third party — this backs `/traces/{trace_id}` regardless of what else is configured. Separately, setting three environment variables ( `LANGSMITH_TRACING=true` , `LANGSMITH_API_KEY` , `LANGSMITH_PROJECT` ) turns on LangSmith's automatic instrumentation for every `LangChain/LangGraph` call, with no code changes — `configure_langsmith()` just sets those env vars from `Settings` before the app is created.

**Q: Why build a real MCP server instead of stubbing the tool calls?** A: Because the whole point of demonstrating MCP is showing tool discovery and invocation as a protocol, not as an in-process function call. A stub would have hidden exactly the part that's interesting to an FDE interviewer: how do you expose actions from a system you don't own to an agent that needs to call them.

## Quick-Reference Cheat Sheet

## Quick-Reference Cheat Sheet

One line each — use these if a term comes up and you need to define it fast, not as a script to recite.

- **RAG (Retrieval-Augmented Generation):** answer a question by first fetching relevant text from your own data, then handing that text to an LLM as context, instead of relying on the model's training data alone.
- **LangGraph:** a library for defining an agent's control flow as an explicit graph ( `StateGraph` ) of nodes and edges, rather than a single prompt loop — supports conditional routing, state passed between steps, and visualization.
- **MCP (Model Context Protocol):** an open protocol for exposing tools/actions to an LLM agent in a standardized way, so the agent can discover and call tools that live in a separate process or system rather than hardcoded functions.
- **Guardrails:** deterministic checks (regex, keyword, threshold-based) that run before the LLM sees input and after it produces output, to block unsafe input or low-confidence/unsupported output — not something the LLM decides for itself.

- **Observability / Tracing:** recording what happened at each step of a request (which node ran, what it returned, how long it took) so any individual request can be replayed and debugged after the fact.
- **FastAPI:** a Python web framework, async-native, that generates request/response validation and interactive API docs (Swagger UI at `/docs`) from Pydantic type hints.
- **ChromaDB:** an open-source vector database; stores text chunks as embeddings and supports similarity search, either embedded in-process or as a separate client/server deployment.
- **LangSmith:** LangChain's hosted tracing/observability product; auto-instruments LangChain/LangGraph calls once a few environment variables are set, no manual instrumentation needed.
- **ReAct agent:** an agent pattern that interleaves reasoning ("what should I do next") with tool calls ("acting"), looping until it decides it has a final answer — used here for the action agent via `langgraph.prebuilt.create_react_agent`.
- **Embedding:** a numeric vector representation of text such that semantically similar text ends up close together in vector space — what makes similarity search possible.
- **Intent classification:** deciding which downstream path (retrieval, summarize, or action) a user's query should take — done here via keyword matching, not a separate ML model.
- **Confidence score:** a numeric estimate of how trustworthy a retrieved answer is, here derived from vector similarity distance ( $1 / (1 + \text{distance})$ ), used to gate auto-answering.